

INFO145 - Diseño y Análisis de Algoritmos

Análisis Amortizado

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Análisis Amortizado

- Técnica que garantiza obtener del costo promedio de una operación en el peor caso.
- Considera que hay operaciones que en el peor caso son muy costosas, mientras que otras, en el mismo peor caso, son mucho mas económicas.
- Pretendemos encontrar una medida justa que nos permita multiplicar el peor costo posible por las n operaciones realizadas.
- Diremos que las operaciones mas baratas nos dejan un "*saldo a favor*" que podremos utilizar para **amortizar** el costo de las operaciones más costosas.

Objetivo principal: Demostrar que el costo promedio por operación en una secuencia es bajo, aunque operaciones individuales puedan ser costosas.

Análisis Amortizado

Diferencia con otros análisis:

Tipo de Análisis	Enfoque	Garantías
Peor caso individual	Operación única	Cota por operación
Caso promedio	Promedio estadístico	No garantiza peor caso
Amortizado	Secuencia completa	Cota total para n operaciones

Análisis Amortizado - Análisis Global

- Estima que todas las operaciones tienen un mismo costo amortizado.
- Considera que, para todo n , una secuencia de n operaciones tiene un costo de $T(n)$ en el peor caso.
- $\Rightarrow$ en el peor caso el costo promedio o costo amortizado por operación es $T(n)/n$, independientemente si algunas operaciones son más costosas que otras.

Análisis Global - Contador Binario

- Se tiene un espacio de k bits para realizar un conteo incremental, de uno en uno, a partir de un valor inicial o estado inicial.
- Sea A un arreglo de k celdas, tal que un número x , de k bits, se almacenana en A con el bit menos significativo en la extrema derecha y el más significativo en la extrema izquierda.

Eje. $x = 35$, $k = 6$

$\Rightarrow A[] = \{100011\}$

$$\text{Así: } x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$$

Counter	Update cost
100011	(initial state)
100100	3
100101	1
100110	2
100111	1
101000	4
101001	1
101010	2
101011	1
101100	3
101101	1

Contador Binario - Algoritmo

El siguiente algoritmo incrementa la cifra de A en uno, recorriendo A de derecha a izquierda.

Input: El arreglo $A[0..k-1]$ en su estado inicial

Output: El mismo arreglo incrementado en 1

INCREMENT($A[0..k-1]$)

$i = k - 1$

 while ($i \geq 0 \wedge A[i] == 1$) do {

$A[i] = 0$

$i = i - 1$

 }

 if($i \geq 0$) then

$A[i] = 1$

}

Contador Binario - Algoritmo

El siguiente algoritmo incrementa la cifra de A en uno, recorriendo A de derecha a izquierda.

Input: El arreglo $A[0..k-1]$ en su estado inicial

Output: El mismo arreglo incrementado en 1

INCREMENT($A[0..k-1]$)

$i = k - 1$

 while ($i \geq 0 \wedge A[i] == 1$) do {

$A[i] = 0$

$i = i - 1$

 }

 if($i \geq 0$) then

$A[i] = 1$

}

Ahora, si quisieramos formar el número n , sumando de uno en uno a partir de 0, se requieren n llamadas a INCREMENT, donde $k = \log(n)$.

$\Rightarrow$ El costo total viene a ser $n \cdot T(k) = n \cdot O(k) = O(n \lg n)$.

Contador Binario - Algoritmo

- Análisis Tradicional de INCREMENT:

El costo total de n llamadas a INCREMENT, para pasar del valor 0 a $n - 1$, es de $n \cdot O(k) = O(n \lg n)$ en el peor caso, con $k = \log n$.

- Análisis Amortizado de INCREMENT:

Si observamos que cada vez que entra al while, se cambia un bit de 1 a 0, y dentro del if se cambia el bit más significativo de 0 a 1.

Contador Binario - Algoritmo

- Análisis Tradicional de INCREMENT:

El costo total de n llamadas a INCREMENT, para pasar del valor 0 a $n - 1$, es de $n \cdot O(k) = O(n \lg n)$ en el peor caso, con $k = \log n$.

- Análisis Amortizado de INCREMENT:

Si observamos que cada vez que entra al while, se cambia un bit de 1 a 0, y dentro del if se cambia el bit más significativo de 0 a 1.

⇒ **Podemos considerar que la complejidad del algoritmo es equivalente a la cantidad de veces que los bits cambian de valor:**

Contador Binario - Algoritmo

- Análisis Tradicional de INCREMENT:

El costo total de n llamadas a INCREMENT, para pasar del valor 0 a $n - 1$, es de $n \cdot O(k) = O(n \lg n)$ en el peor caso, con $k = \log n$.

- Análisis Amortizado de INCREMENT:

Si observamos que cada vez que entra al while, se cambia un bit de 1 a 0, y dentro del if se cambia el bit más significativo de 0 a 1.

⇒ **Podemos considerar que la complejidad del algoritmo es equivalente a la cantidad de veces que los bits cambian de valor:**

- El bit 0 (de derecha a izquierda) cambia en todas las llamadas a INCREMENT
- El bit 1 cambia $\lfloor n/2 \rfloor$ veces en las n llamadas a INCREMENT
- El bit 2 cambia $\lfloor n/4 \rfloor$ veces en las n llamadas a INCREMENT

...

- El bit $k - 1$ (extrema izquierda) cambia $\lfloor n/2^{k-1} \rfloor$ veces en las n llamadas a INCREMENT

∴ Para $i = 0 \dots k - 1$ hay $\lfloor n/2^i \rfloor$ cambios en las n llamadas a INCREMENT.

Contador Binario - Algoritmo

- Análisis Tradicional de INCREMENT:

El costo total de n llamadas a INCREMENT, para pasar del valor 0 a $n - 1$, es de $n \cdot O(k) = O(n \lg n)$ en el peor caso, con $k = \log n$.

- Análisis Amortizado de INCREMENT:

Si observamos que cada vez que entra al while, se cambia un bit de 1 a 0, y dentro del if se cambia el bit más significativo de 0 a 1.

⇒ **Podemos considerar que la complejidad del algoritmo es equivalente a la cantidad de veces que los bits cambian de valor:**

- El bit 0 (de derecha a izquierda) cambia en todas las llamadas a INCREMENT
- El bit 1 cambia $\lfloor n/2 \rfloor$ veces en las n llamadas a INCREMENT
- El bit 2 cambia $\lfloor n/4 \rfloor$ veces en las n llamadas a INCREMENT

...

- El bit $k - 1$ (extrema izquierda) cambia $\lfloor n/2^{k-1} \rfloor$ veces en las n llamadas a INCREMENT

∴ Para $i = 0 \dots k - 1$ hay $\lfloor n/2^i \rfloor$ cambios en las n llamadas a INCREMENT.

⇒ El costo total es: $T(n) = \sum_{i=0}^{k-1} \lfloor \frac{n}{2^i} \rfloor$

Contador Binario - Algoritmo

- Análisis Tradicional de INCREMENT:

El costo total de n llamadas a INCREMENT, para pasar del valor 0 a $n - 1$, es de $n \cdot O(k) = O(n \lg n)$ en el peor caso, con $k = \log n$.

- Análisis Amortizado de INCREMENT:

Si observamos que cada vez que entra al while, se cambia un bit de 1 a 0, y dentro del if se cambia el bit más significativo de 0 a 1.

⇒ **Podemos considerar que la complejidad del algoritmo es equivalente a la cantidad de veces que los bits cambian de valor:**

- El bit 0 (de derecha a izquierda) cambia en todas las llamadas a INCREMENT
- El bit 1 cambia $\lfloor n/2 \rfloor$ veces en las n llamadas a INCREMENT
- El bit 2 cambia $\lfloor n/4 \rfloor$ veces en las n llamadas a INCREMENT

...

- El bit $k - 1$ (extrema izquierda) cambia $\lfloor n/2^{k-1} \rfloor$ veces en las n llamadas a INCREMENT

∴ Para $i = 0 \dots k - 1$ hay $\lfloor n/2^i \rfloor$ cambios en las n llamadas a INCREMENT.

⇒ El costo total es:
$$T(n) = \sum_{i=0}^{k-1} \lfloor \frac{n}{2^i} \rfloor \leq n \sum_{i=0}^{k-1} \frac{1}{2^i} \leq 2n = O(n)$$

Contador Binario - Algoritmo

- Análisis Tradicional de INCREMENT:

El costo total de n llamadas a INCREMENT, para pasar del valor 0 a $n - 1$, es de $n \cdot O(k) = O(n \lg n)$ en el peor caso, con $k = \log n$.

- Análisis Amortizado de INCREMENT:

Si observamos que cada vez que entra al while, se cambia un bit de 1 a 0, y dentro del if se cambia el bit más significativo de 0 a 1.

⇒ **Podemos considerar que la complejidad del algoritmo es equivalente a la cantidad de veces que los bits cambian de valor:**

- El bit 0 (de derecha a izquierda) cambia en todas las llamadas a INCREMENT
- El bit 1 cambia $\lfloor n/2 \rfloor$ veces en las n llamadas a INCREMENT
- El bit 2 cambia $\lfloor n/4 \rfloor$ veces en las n llamadas a INCREMENT

...

- El bit $k - 1$ (extrema izquierda) cambia $\lfloor n/2^{k-1} \rfloor$ veces en las n llamadas a INCREMENT

∴ Para $i = 0 \dots k - 1$ hay $\lfloor n/2^i \rfloor$ cambios en las n llamadas a INCREMENT.

⇒ El costo total es: $T(n) = \sum_{i=0}^{k-1} \lfloor \frac{n}{2^i} \rfloor \leq n \sum_{i=0}^{k-1} \frac{1}{2^i} \leq 2n = O(n)$

⇒ Costo promedio por operación = $O(n)/n = O(1)$

Contador Binario - Ejemplo

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Con análisis tradicional da un costo de $T(n) = n \cdot O(k)$ en el peor caso, mientras que con análisis amortizados el costo más preciso es $2n$.

Análisis Amortizado - Contabilidad de Costo

- Asignamos diferentes costos a diferentes operaciones. Es distinto al método anterior, ya que en el análisis global a todas las operaciones le asignamos un mismo costo $T(n)/n$.
- De este modo, cada vez que asignamos un mayor costo que el real a una operación, tendremos CRÉDITO para luego pagar costos más caros en otras operaciones.
- Se debe asegurar que el costo amortizado de una secuencia de operaciones cualquiera sea siempre una cota superior al problema.

Sean:

c_i : costo real de la operación i

$\hat{c}_i$: costo amortizado de la operación i .

$\Rightarrow$ se debe asegurar que $\sum_{i=0}^n \hat{c}_i \geq \sum_{i=0}^n c_i$

Así, el crédito a favor es $\sum_{i=0}^n \hat{c}_i - \sum_{i=0}^n c_i$

Contabilidad de Costo - Contador Binario

- Sea el costo amortizado $\hat{c}_i = 2m$, de cambiar m bits de 0 a 1, para un contador binario que comienza con todos sus bits en 0.
- $\Rightarrow$ esto acumula m bits a nuestro crédito, los cuales usaremos para pagar el costo de cambiar estos mismos bits de 1 a 0.

Contabilidad de Costo - Contador Binario

- Sea el costo amortizado $\hat{c}_i = 2m$, de cambiar m bits de 0 a 1, para un contador binario que comienza con todos sus bits en 0.
- $\Rightarrow$ esto acumula m bits a nuestro crédito, los cuales usaremos para pagar el costo de cambiar estos mismos bits de 1 a 0.
- Si el contador comienza en cero (las k celdas en 0), entonces la primera operación, de cambiar de 0 a 1, aumenta el costo en 2 por cada bit que se cambia. Si el contador comienza en un valor $u > 0$, podríamos considerar un costo extra inicial $\leq \lg u \leq k = \lg n = o(n)$ (como máximo).
- De esta forma nuestro crédito nunca será negativo y los cambios de 1 a 0 se pagaran con el crédito a favor que se tiene.

Contabilidad de Costo - Contador Binario

- Note que en el algoritmo INCREMENT(), el cambio de 0 a 1 solo se realiza una vez por cada llamada (al final)
- Si el contador comienza de 0 $\Rightarrow$ para n llamadas a INCREMENT, tenemos que: $T(n) = 2n$
- Si el contador comienza de un valor $u > 0 \Rightarrow$ para n llamadas a INCREMENT, tenemos que: $T(n) = 2n + \log u = O(n)$